Process Synchronization

Hands-On Lab Report (Hands-On 1, 2 and 3)

Name	Rohit Raghuwanshi
Roll No.	12341820
Course Code	CSL301
Date	13 October 2026
Machine	MacBook Air (Apple M2, arm64), macOS
Compiler	Apple clang 21.0.0 (invoked as gcc), flags: -O0 -Wall

Contents of this report: for every question, the commands used, the source code, the observed output with screenshots, and the answer. Source files are in the code folder of each Hands-On directory and screenshots in its screenshots folder.

Setup

Working folder: ~/os_lab/sync. The two supplied programs (race_condition.c and peterson_processes.c) were created in this folder.

```
mkdir -p ~/os_lab/sync && cd ~/os_lab/sync
ls -l
wc -l race_condition.c peterson_processes.c
gcc --version
```

```
❯ ls -l
wc -l race_condition.c peterson_processes.c
total 16
-rw-r--r--  1 rohitrg  staff  1902 13 Oct 13:16 peterson_processes.c
-rw-r--r--  1 rohitrg  staff  1343 13 Oct 13:15 race_condition.c
      60 race_condition.c
      84 peterson_processes.c
     144 total
```

Figure 1: Both supplied source files present in the working folder

Hands-On 1: Run and investigate the race-condition program

Program: race_condition.c. Two processes (parent and child created with fork) increment a shared counter in anonymous shared memory (mmap) without any lock. Each process does temp = counter, waits DELAY_US = 10 microseconds, then writes counter = temp + 1. Expected total = 2 x ITERATIONS.

Source code: race_condition.c

```
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <unistd.h>

#define ITERATIONS 1000
#define DELAY_US 10

typedef struct {
    volatile int counter;
} SharedData;

static void increment_without_lock(SharedData *sh) {
    for (int i = 0; i < ITERATIONS; ++i) {
        int temp = sh->counter;
        usleep(DELAY_US);  /* Makes interleaving easier to observe */
        sh->counter = temp + 1;
    }
}

int main(void) {
    SharedData *sh = mmap(NULL, sizeof(*sh),
                          PROT_READ | PROT_WRITE,
                          MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    if (sh == MAP_FAILED) {
        perror("mmap");
        return EXIT_FAILURE;
    }

    sh->counter = 0;

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        munmap(sh, sizeof(*sh));
        return EXIT_FAILURE;
    }

    if (pid == 0) {
        increment_without_lock(sh);
        munmap(sh, sizeof(*sh));
        _exit(EXIT_SUCCESS);
    }

    increment_without_lock(sh);

    if (waitpid(pid, NULL, 0) < 0) {
        perror("waitpid");
        munmap(sh, sizeof(*sh));
        return EXIT_FAILURE;
```

```
    }

    printf("Expected counter: %d\n", 2 * ITERATIONS);
    printf("Observed counter: %d\n", sh->counter);

    munmap(sh, sizeof(*sh));
    return EXIT_SUCCESS;
}
```

Q1. Compile and run three times without changing the source

```
gcc -O0 -Wall -o race race_condition.c
./race
./race
./race
```

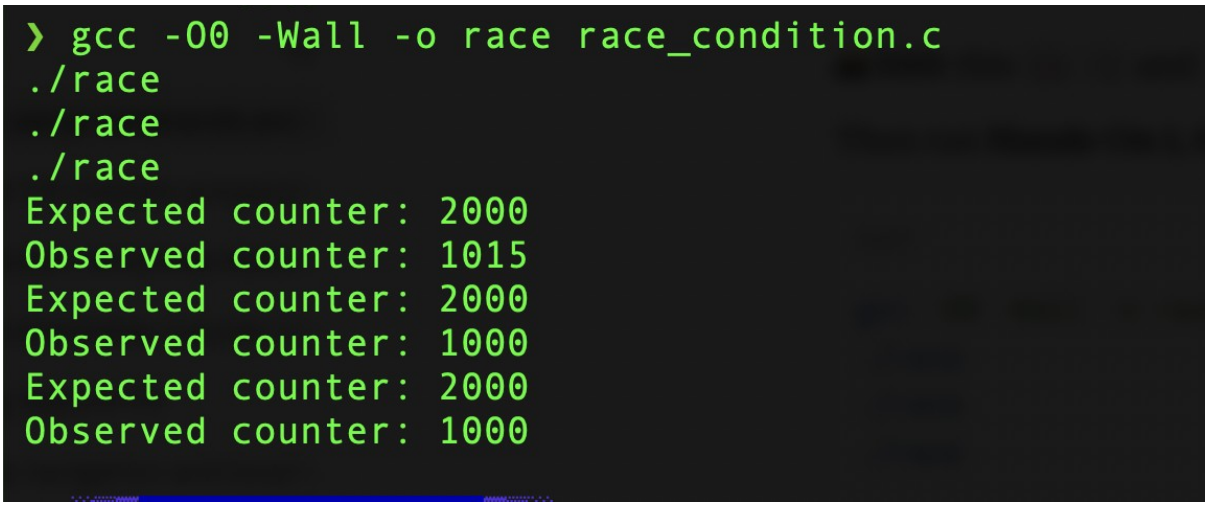


Figure 2: race_condition.c, ITERATIONS = 1000, three runs

Run	Iterations/process	Expected total	Observed total
1	1000	2000	1015
2	1000	2000	1000
3	1000	2000	1000

Every run lost about half of the increments. Both processes read the same value, sleep, and then write back the same result, so one update overwrites the other (lost update).

Q2. ITERATIONS changed to 100 (in a copy)

```
sed 's/#define ITERATIONS 1000/#define ITERATIONS 100/' race_condition.c > race_100.c
grep ITERATIONS race_100.c | head -1
gcc -O0 -Wall -o race_100 race_100.c
./race_100
./race_100
./race_100
```

```
❯ sed 's/#define ITERATIONS 1000/#define ITERATIONS 100/' race_condition.c > rac
e_100.c
grep ITERATIONS race_100.c | head -1
gcc -O0 -Wall -o race_100 race_100.c
./race_100
./race_100
./race_100
#define ITERATIONS 100
Expected counter: 200
Observed counter: 123
Expected counter: 200
Observed counter: 110
Expected counter: 200
Observed counter: 107
```

Figure 3: race_100.c (ITERATIONS = 100), three runs

Run	Iterations/process	Expected total	Observed total
1	100	200	123
2	100	200	110
3	100	200	107

Q3. Delay between read and write removed (in a copy)

The line usleep(DELAY_US) was deleted in a copy, race_nodelay.c. The grep prints nothing, which confirms the delay is gone.

```
sed '/usleep(DELAY_US)/d' race_condition.c > race_nodelay.c
grep -n usleep race_nodelay.c
gcc -O0 -Wall -o race_nodelay race_nodelay.c
for i in 1 2 3 4 5; do ./race_nodelay; done
```

Terminal output for N = 1000 (no delay):

```
Expected counter: 2000
Observed counter: 2000
(the same two lines appeared for all 5 runs: all 5 runs gave 2000)
```

Because the read-to-write window is now only a few instructions long and each process finishes very quickly, the two processes barely overlap. The workload was then increased to 1,000,000 per process to give them time to collide:

```
sed -e '/usleep(DELAY_US)/d' -e 's/#define ITERATIONS 1000/#define ITERATIONS 1000000/'
race_condition.c > race_nodelay_big.c
gcc -O0 -Wall -o race_nodelay_big race_nodelay_big.c
for i in 1 2 3; do ./race_nodelay_big; done
```

```
❯ sed -e '/usleep(DELAY_US)/d' -e 's/#define ITERATIONS 1000/#define ITERATIONS
1000000/' race_condition.c > race_nodelay_big.c
gcc -O0 -Wall -o race_nodelay_big race_nodelay_big.c
for i in 1 2 3; do ./race_nodelay_big; done
Expected counter: 2000000
Observed counter: 1105557
Expected counter: 2000000
Observed counter: 1044718
Expected counter: 2000000
Observed counter: 1027103
```

Figure 4: no delay, ITERATIONS = 1,000,000, three runs

Q4. Table of all observed values (every run reported)

Version	N per process	Expected	Observed
Original, run 1	1000	2000	1015
Original, run 2	1000	2000	1000
Original, run 3	1000	2000	1000
N = 100, run 1	100	200	123
N = 100, run 2	100	200	110
N = 100, run 3	100	200	107
No delay, runs 1-5	1000	2000	2000 (all five runs)
No delay, run 1	1,000,000	2,000,000	1105557
No delay, run 2	1,000,000	2,000,000	1044718
No delay, run 3	1,000,000	2,000,000	1027103

Q5. Does one correct run prove that the program is race-free?

Answer: No. A single correct run only shows that one particular interleaving happened to be harmless: my no-delay program printed 2000 five times at N = 1000, yet the identical code lost almost half of the updates at N = 1,000,000. Race conditions depend on timing, so only reasoning about all possible interleavings, not testing a few runs, can show a program is race-free.

Hands-On 2: Implement and trace three approaches

Q6-7. Three separate files, with turn and flag[2] added before fork()

The original race_condition.c was kept unchanged. The three versions were built from a common header (head.txt, with the shared structure containing counter, turn and flag[2]) and a common main (main.txt, which initialises all fields before fork()), plus a different worker() function for each version.

```
❯ ls -l head.txt main.txt
wc -l head.txt main.txt
-rw-r--r--  1 rohitrg  staff  280 13 Oct 13:18 head.txt
-rw-r--r--  1 rohitrg  staff  842 13 Oct 13:19 main.txt
      16 head.txt
      32 main.txt
      48 total
```

Figure 5: head.txt and main.txt created

Common header (head.txt)

```
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <unistd.h>

#define ITERATIONS 1000
#define DELAY_US 10

typedef struct {
    volatile int counter;
    volatile int turn;
    volatile int flag[2];
} SharedData;
```

Common main (main.txt): initialisation is done before fork()

```
int main(void) {
    SharedData *sh = mmap(NULL, sizeof(*sh),
                          PROT_READ | PROT_WRITE,
                          MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    if (sh == MAP_FAILED) { perror("mmap"); return EXIT_FAILURE; }

    /* Initialize BEFORE fork() */
    sh->counter = 0;
    sh->turn = 0;
    sh->flag[0] = 0;
    sh->flag[1] = 0;

    pid_t pid = fork();
    if (pid < 0) { perror("fork"); munmap(sh, sizeof(*sh)); return EXIT_FAILURE; }

    if (pid == 0) {
        worker(sh, 1);
        munmap(sh, sizeof(*sh));
        _exit(EXIT_SUCCESS);
    }
```

```
    worker(sh, 0);

    if (waitpid(pid, NULL, 0) < 0) { perror("waitpid"); return EXIT_FAILURE; }

    printf("Expected counter: %d\n", 2 * ITERATIONS);
    printf("Observed counter: %d\n", sh->counter);

    munmap(sh, sizeof(*sh));
    return EXIT_SUCCESS;
}
```

Q8. Version A: strict alternation

```
static void worker(SharedData *sh, int me) {
    int other = 1 - me;
    for (int i = 0; i < ITERATIONS; ++i) {
        /* ===== ENTRY SECTION ===== */
        while (sh->turn != me) { }

        /* ===== CRITICAL SECTION ===== */
        int temp = sh->counter;
        usleep(DELAY_US);
        sh->counter = temp + 1;

        /* ===== EXIT SECTION ===== */
        sh->turn = other;
    }
}
```

```
cat head.txt > version_a.c
cat >> version_a.c   # worker() above
cat main.txt >> version_a.c
gcc -O0 -Wall -o version_a version_a.c
./version_a
./version_a
./version_a
```

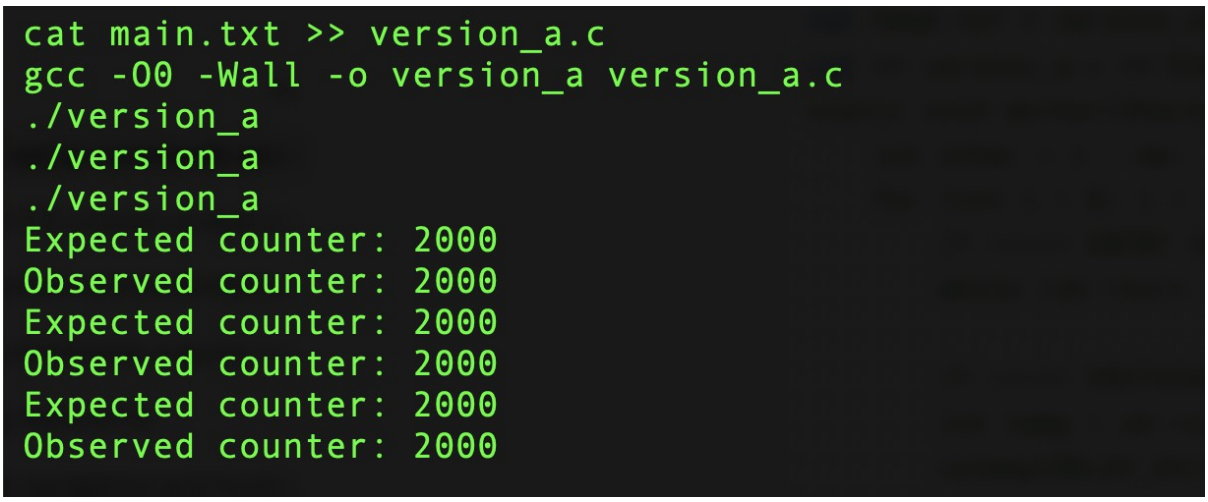


Figure 6: Version A, three runs

Run	Expected	Observed	Notes
1	2000	2000	correct
2	2000	2000	correct
3	2000	2000	correct

Strict alternation guarantees mutual exclusion because only the process whose turn it is can enter. Its weakness is that the processes must take turns: if one stops, the other waits forever even when the critical section is free.

Q9. Version B: check then set (flawed)

```
static void worker(SharedData *sh, int me) {
    int other = 1 - me;
    for (int i = 0; i < ITERATIONS; ++i) {
        /* ===== ENTRY SECTION ===== */
        while (sh->flag[other]) { }
        sh->flag[me] = 1;

        /* ===== CRITICAL SECTION ===== */
        int temp = sh->counter;
        usleep(DELAY_US);
        sh->counter = temp + 1;

        /* ===== EXIT SECTION ===== */
        sh->flag[me] = 0;
    }
}
```

```
gcc -O0 -Wall -o version_b version_b.c
for i in 1 2 3 4 5 6 7 8; do ./version_b; done
./version_b
```

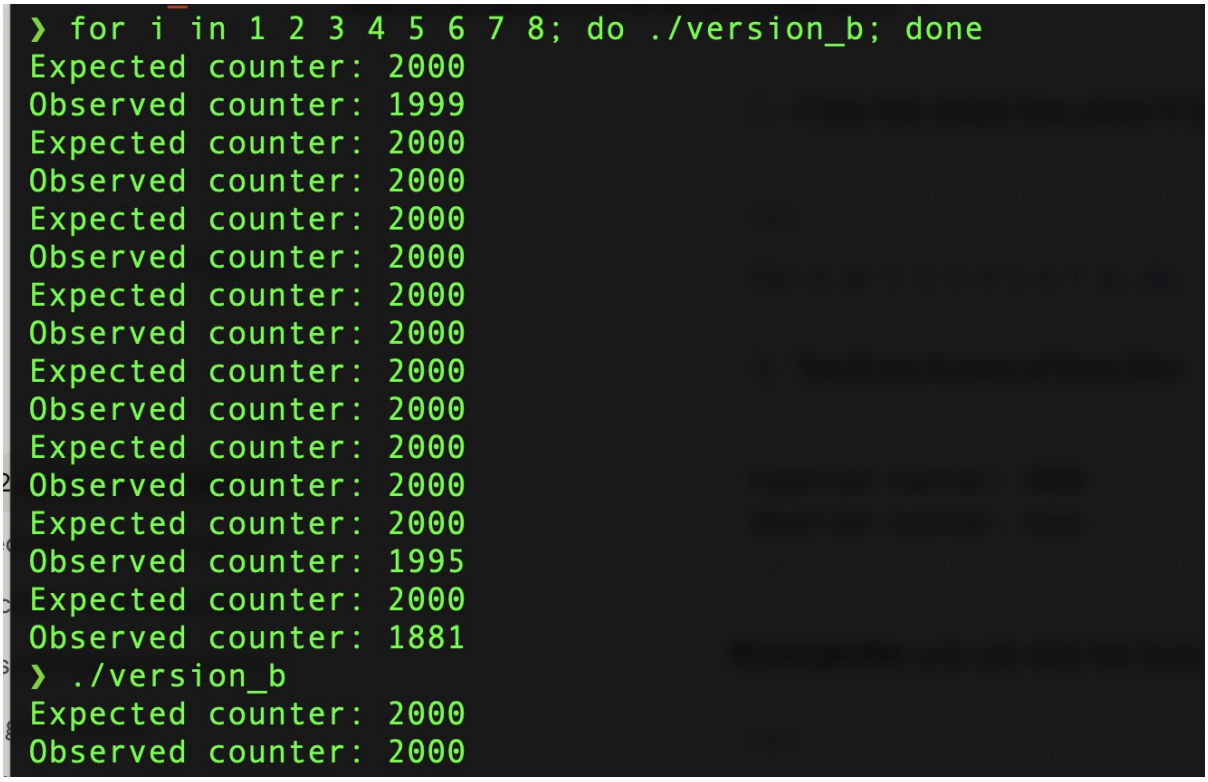


Figure 7: Version B, nine runs

Run	N per process	Expected	Observed	Result
1	1000	2000	1999	failed
2	1000	2000	2000	ok

Run	N per process	Expected	Observed	Result
3	1000	2000	2000	ok
4	1000	2000	2000	ok
5	1000	2000	2000	ok
6	1000	2000	2000	ok
7	1000	2000	1995	failed
8	1000	2000	1881	failed
9	1000	2000	2000	ok

Why success in a single run is not a proof: both processes can pass the check while (flag[other]) before either has set its own flag, because the check and the set are two separate steps. Both then set their flags and enter the critical section together, which breaks mutual exclusion. This only happens with a particular timing, so most runs (6 of 9 here) were correct by luck, while runs 1, 7 and 8 lost updates.

Q10. Version C: set then wait (can deadlock)

```
static void worker(SharedData *sh, int me) {
    int other = 1 - me;
    for (int i = 0; i < ITERATIONS; ++i) {
        /* ===== ENTRY SECTION ===== */
        sh->flag[me] = 1;
        while (sh->flag[other]) { }

        /* ===== CRITICAL SECTION ===== */
        int temp = sh->counter;
        usleep(DELAY_US);
        sh->counter = temp + 1;

        /* ===== EXIT SECTION ===== */
        sh->flag[me] = 0;
    }
}
```

macOS has no timeout command, so a 5-second alarm was used to stop the hang:

```
gcc -O0 -Wall -o version_c version_c.c
perl -e 'alarm 5; exec @ARGV' ./version_c
echo "exit code: $?"
pkill version_c
```

Result: both runs hung and were killed by the alarm after 5 seconds. Exit code 142 = 128 + 14 (SIGALRM). No "Observed counter" line was ever printed. See Figure 8 (top part).

Run	N per process	Expected	Observed
1	1000	2000	hangs, killed by alarm, exit code 142
2	1000	2000	hangs, killed by alarm, exit code 142

Trace of the state flag[0] == 1 and flag[1] == 1:

Step	What happens
1	P0 sets flag[0] = 1.
2	P1 sets flag[1] = 1 before P0 has left its wait loop.

Step	What happens
3	P0 checks flag[1], which is 1, so P0 keeps waiting. P1 checks flag[0], which is 1, so P1 keeps waiting.
4	State: flag[0] == 1 and flag[1] == 1. Each flag is cleared only after that process finishes its critical section, which neither can enter. Nothing can change.

Mutual exclusion is preserved (nobody is in the critical section), but there is no progress. This is a deadlock.

Q11. Entry, critical and exit sections labelled in each source file

Each source file contains the comments ENTRY SECTION, CRITICAL SECTION and EXIT SECTION. They were checked with:

```
grep -n "=====" version_a.c version_b.c version_c.c
```

```
❯ perl -e 'alarm 5; exec @ARGV' ./version_c
echo "exit code: $?"
pkill version_c
[1]    91053 alarm      perl -e 'alarm 5; exec @ARGV' ./version_c
exit code: 142
❯ perl -e 'alarm 5; exec @ARGV' ./version_c; echo "exit code: $?"; pkill version
_c
[1]    91066 alarm      perl -e 'alarm 5; exec @ARGV' ./version_c
exit code: 142
❯ grep -n "=====" version_a.c version_b.c version_c.c
version_a.c:20:        /* ===== ENTRY SECTION ===== */
version_a.c:23:        /* ===== CRITICAL SECTION ===== */
version_a.c:28:        /* ===== EXIT SECTION ===== */
version_b.c:20:        /* ===== ENTRY SECTION ===== */
version_b.c:24:        /* ===== CRITICAL SECTION ===== */
version_b.c:29:        /* ===== EXIT SECTION ===== */
version_c.c:20:        /* ===== ENTRY SECTION ===== */
version_c.c:24:        /* ===== CRITICAL SECTION ===== */
version_c.c:29:        /* ===== EXIT SECTION ===== */
```

Figure 8: Version C hang (exit code 142, two runs) and the grep showing all section labels in all three files

Version	Entry section	Exit section
A: Strict alternation	`while (sh->turn != me) { }`	`sh->turn = other;`
B: Check then set (flawed)	`while (sh->flag[other]) { }` `sh->flag[me] = 1;`	`sh->flag[me] = 0;`
C: Set then wait (can deadlock)	`sh->flag[me] = 1;` `while (sh->flag[other]) { }`	`sh->flag[me] = 0;`

In all three versions the critical section is: temp = sh->counter; usleep(DELAY_US); sh->counter = temp + 1;

Hands-On 3: Modify, run, compare

Class Peterson program (peterson_processes.c)

```
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <unistd.h>

#define ITERATIONS 10000

/*
 * Educational illustration of classic Peterson's Algorithm.
 * IMPORTANT: volatile alone is not a portable synchronization
 * mechanism in modern C. Use a process-shared POSIX mutex or a
 * properly designed platform-supported atomic implementation in
 * production code.
 */
typedef struct {
    volatile int counter;
    volatile int flag[2];
    volatile int turn;
} SharedData;

static void worker(SharedData *sh, int me) {
    int other = 1 - me;

    for (int i = 0; i < ITERATIONS; ++i) {
        /* Entry section */
        sh->flag[me] = 1;
        sh->turn = other;

        while (sh->flag[other] == 1 && sh->turn == other) {
            /* Busy waiting */
        }

        /* Critical section */
        int temp = sh->counter;
        sh->counter = temp + 1;

        /* Exit section */
        sh->flag[me] = 0;
    }
}

int main(void) {
    SharedData *sh = mmap(NULL, sizeof(*sh),
                          PROT_READ | PROT_WRITE,
                          MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    if (sh == MAP_FAILED) {
        perror("mmap");
        return EXIT_FAILURE;
    }

    sh->counter = 0;
    sh->flag[0] = 0;
    sh->flag[1] = 0;
    sh->turn = 0;
```

```
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        munmap(sh, sizeof(*sh));
        return EXIT_FAILURE;
    }

    if (pid == 0) {
        worker(sh, 1);
        munmap(sh, sizeof(*sh));
        _exit(EXIT_SUCCESS);
    }

    worker(sh, 0);

    if (waitpid(pid, NULL, 0) < 0) {
        perror("waitpid");
        munmap(sh, sizeof(*sh));
        return EXIT_FAILURE;
    }

    printf("Expected counter: %d\n", 2 * ITERATIONS);
    printf("Observed counter: %d\n", sh->counter);

    munmap(sh, sizeof(*sh));
    return EXIT_SUCCESS;
}
```

Q12. Compile and run the class Peterson program

```
gcc -O0 -Wall -o peterson peterson_processes.c
./peterson
./peterson
./peterson
```

```
❯ cd ~/os_lab/sync
ls -l peterson_processes.c
-rw-r--r--  1 rohitrg  staff  1902 13 Oct 13:16 peterson_processes.c
❯ gcc -O0 -Wall -o peterson peterson_processes.c
./peterson
./peterson
./peterson
Expected counter: 20000
Observed counter: 20000
Expected counter: 20000
Observed counter: 20000
Expected counter: 20000
Observed counter: 20000
```

Figure 9: Peterson, N = 10000, three runs

Run	N per process	Expected	Observed
1	10000	20000	20000
2	10000	20000	20000
3	10000	20000	20000

Q13. 100 increments per process

```
sed 's/#define ITERATIONS 10000/#define ITERATIONS 100/' peterson_processes.c >
peterson_100.c
grep ITERATIONS peterson_100.c | head -1
gcc -O0 -Wall -o peterson_100 peterson_100.c
for i in 1 2 3; do ./peterson_100; done
```

Q14. Larger workload (1,000,000 per process)

```
sed 's/#define ITERATIONS 10000/#define ITERATIONS 1000000/' peterson_processes.c >
peterson_big.c
gcc -O0 -Wall -o peterson_big peterson_big.c
for i in 1 2 3; do ./peterson_big; done
```

```
> peterson_100.c
grep ITERATIONS peterson_100.c | head -1
gcc -O0 -Wall -o peterson_100 peterson_100.c
for i in 1 2 3; do ./peterson_100; done
#define ITERATIONS 100
Expected counter: 200
Observed counter: 200
Expected counter: 200
Observed counter: 200
Expected counter: 200
Observed counter: 200
❯ sed 's/#define ITERATIONS 10000/#define ITERATIONS 1000000/' peterson_processe
s.c > peterson_big.c
gcc -O0 -Wall -o peterson_big peterson_big.c
for i in 1 2 3; do ./peterson_big; done
Expected counter: 2000000
Observed counter: 2000000
Expected counter: 2000000
Observed counter: 2000000
Expected counter: 2000000
Observed counter: 2000000
```

Figure 10: Peterson with N = 100 and N = 1,000,000, three runs each

Run	N per process	Expected	Observed
1-3	100	200	200, 200, 200
1-3	1,000,000	2,000,000	2000000, 2000000, 2000000

Peterson was correct in every run on this machine. This is evidence, not proof: plain volatile does not insert memory barriers, and Apple Silicon (ARM64) has a weak memory model, so the code is not guaranteed to be correct on all hardware (the comment in the supplied source says the same).

Q15. Peterson waiting condition removed (in a copy)

In a copy, the waiting condition was replaced by while (0 /* WAIT DISABLED */). The original file was never edited, so "restoring" it is shown by the diff, which lists only line 31.

```
perl -pe 's/while \(sh->flag\[other\] == 1 && sh->turn == other\)/while (0 \/* WAIT
DISABLED *\/)/' peterson_processes.c > peterson_nowait.c
grep -n "WAIT DISABLED" peterson_nowait.c
gcc -O0 -Wall -o peterson_nowait peterson_nowait.c
for i in 1 2 3; do ./peterson_nowait; done
sed 's/#define ITERATIONS 10000/#define ITERATIONS 1000000/' peterson_nowait.c >
peterson_nowait_big.c
gcc -O0 -o peterson_nowait_big peterson_nowait_big.c
for i in 1 2 3; do ./peterson_nowait_big; done
```

```
diff peterson_processes.c peterson_nowait.c
```

```
❯ gcc -O0 -Wall -o peterson_nowait peterson_nowait.c
for i in 1 2 3; do ./peterson_nowait; done
Expected counter: 20000
Observed counter: 20000
Expected counter: 20000
Observed counter: 20000
Expected counter: 20000
Observed counter: 20000
❯ sed 's/#define ITERATIONS 10000/#define ITERATIONS 1000000/' peterson_nowait.c
 > peterson_nowait_big.c
gcc -O0 -o peterson_nowait_big peterson_nowait_big.c
for i in 1 2 3; do ./peterson_nowait_big; done
Expected counter: 2000000
Observed counter: 1119496
Expected counter: 2000000
Observed counter: 1003529
Expected counter: 2000000
Observed counter: 1040750
```

Figure 11: Peterson without the wait, N = 10000 and N = 1,000,000

Terminal output of the grep and diff commands:

```
$ grep -n "WAIT DISABLED" peterson_nowait.c
31:        while (0 /* WAIT DISABLED */) {

$ diff peterson_processes.c peterson_nowait.c
31c31
<         while (sh->flag[other] == 1 && sh->turn == other) {
---
>         while (0 /* WAIT DISABLED */) {
```

Version	N per process	Expected	Observed
No wait, runs 1-3	10000	20000	20000, 20000, 20000
No wait, run 1	1,000,000	2,000,000	1119496
No wait, run 2	1,000,000	2,000,000	1003529
No wait, run 3	1,000,000	2,000,000	1040750

Without the wait, nearly half of the updates were lost at N = 1,000,000, while the same copy happened to be correct at N = 10000. The real Peterson program gave the exact total in all runs at N = 1,000,000, so the waiting condition is what provides mutual exclusion.

Q16. Peterson compared with the race-condition program at the same N

```
sed 's/#define ITERATIONS 10000/#define ITERATIONS 1000/' peterson_processes.c >
peterson_1000.c
gcc -O0 -o peterson_1000 peterson_1000.c
echo "--- Peterson N=1000 ---"; for i in 1 2 3; do ./peterson_1000; done
echo "--- Race N=1000 ---"; for i in 1 2 3; do ./race; done
```

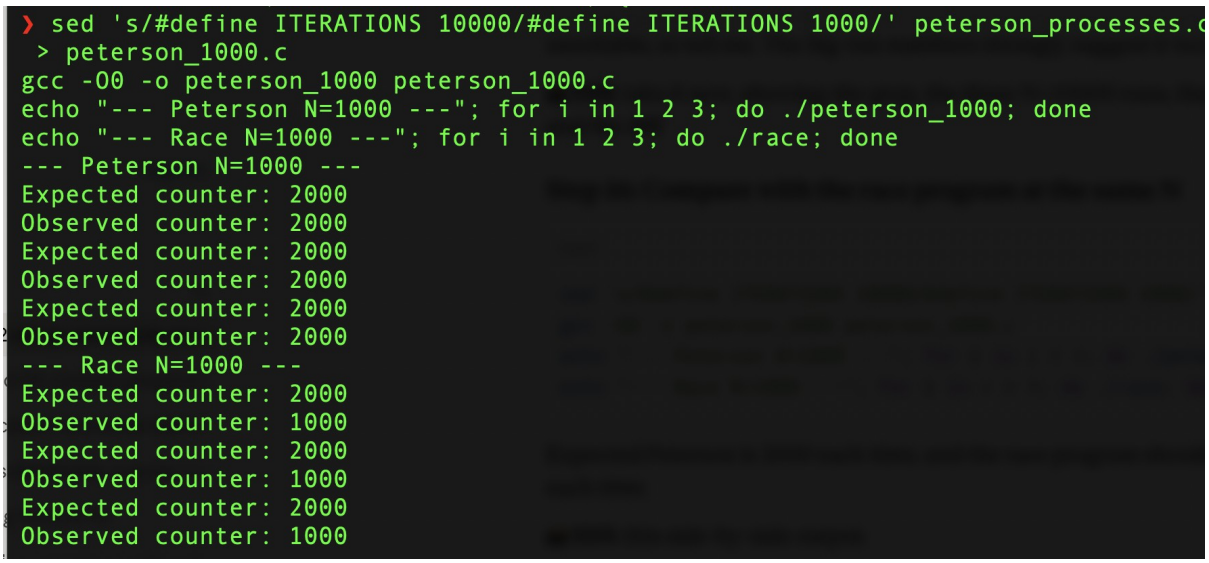


Figure 12: Peterson and the race-condition program, both with N = 1000

Program	N per process	Expected	Observed
Peterson, runs 1-3	1000	2000	2000, 2000, 2000
Race condition, runs 1-3	1000	2000	1000, 1000, 1000

With the same number of increments, Peterson kept every update while the unprotected program lost about half. The entry and exit sections let only one process into the critical section at a time.

Q17. Submitted files, results table and explanation

Submitted source files: race_condition.c and peterson_processes.c (HandsOn1_Race_Condition/code and HandsOn3_Peterson/code), together with all modified copies. Final table of expected and observed results:

Program/version	N per process	Expected	Observed	Notes
Race condition, run 1	1000	2000	1015	lost updates
Race condition, run 2	1000	2000	1000	lost updates
Race condition, run 3	1000	2000	1000	lost updates
Race, N = 100	100	200	123, 110, 107	lost updates
Race, no delay	1000	2000	2000 (x5)	correct by luck
Race, no delay	1,000,000	2,000,000	1105557, 1044718, 1027103	lost updates
Strict alternation	1000	2000	2000 (x3)	correct, forced turns
Check-then-set flags	1000	2000	1999, 2000 (x6), 1995, 1881	rare failures
Set-then-wait flags	1000	2000	hangs (exit 142)	deadlock
Peterson	100	200	200 (x3)	correct
Peterson	1000	2000	2000 (x3)	correct
Peterson	10000	20000	20000 (x3)	correct
Peterson	1,000,000	2,000,000	2000000 (x3)	correct

Program/version	N per process	Expected	Observed	Notes
Peterson without wait (copy)	10000	20000	20000 (x3)	correct by luck
Peterson without wait (copy)	1,000,000	2,000,000	1119496, 1003529, 1040750	lost updates

Why output alone is not a complete correctness proof: output only shows the interleavings that happened in the runs I tried, and a race can hide in a timing that rarely occurs. In my experiments the no-delay race program and the Peterson copy without its wait both printed the exact expected value at small N but lost nearly half of the updates at N = 1,000,000, and Version B gave 2000 in most runs yet failed in others. Correctness needs an argument over all interleavings (mutual exclusion, progress and bounded waiting), and on modern CPUs such as the M2 it also needs memory-ordering guarantees, which plain volatile does not provide. So Peterson passing every run here is evidence, not proof.